Computer Architecture

Lab 1

Register File Implementation

Prepared by

Omar Khaled Hussien El-sayed 22010962
Yaseen Asaad Ahmed Farid 22011349

1. Code

✓ Main logic

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RegisterFile is
    Port (
        ReadReg1  : in  std_logic_vector(4 downto 0);
        ReadReg2  : in  std_logic_vector(4 downto 0);
        WriteReg  : in  std_logic_vector(4 downto 0);
        WriteData : in  std_logic_vector(31 downto 0);
        clk       : in  std_logic;
        RegWrite  : in  std_logic;
        ReadData1 : out std_logic_vector(31 downto 0);
        ReadData2 : out std_logic_vector(31 downto 0)
    );
end RegisterFile;

architecture Behavioral of RegisterFile is

type reg_file is array (31 downto 0) of std_logic_vector(31 downto 0);
signal Registers : reg_file := (others => (others => '0'));

begin

-- WRITE OPERATION
process(clk)
begin
    if rising_edge(clk) then
        if RegWrite = '1' then
          if to_integer(unsigned(Writereg)) /= 0 then
            Registers(to_integer(unsigned(WriteReg))) <= WriteData;
          end if;
        end if;
    end if;
end process;
-- READ OPERATION
process(clk)
begin
    if falling_edge(clk) then
        ReadData1 <= Registers(to_integer(unsigned(ReadReg1)));
        ReadData2 <= Registers(to_integer(unsigned(ReadReg2)));
    end if;
end process;
```

✓ Test bench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_RegisterFile is
end tb_RegisterFile;

architecture behavior of tb_RegisterFile is
    component RegisterFile
    Port (
        ReadReg1  : in  STD_LOGIC_VECTOR (4 downto 0);
        ReadReg2  : in  STD_LOGIC_VECTOR (4 downto 0);
        WriteReg  : in  STD_LOGIC_VECTOR (4 downto 0);
        WriteData : in  STD_LOGIC_VECTOR (31 downto 0);
        clk       : in  STD_LOGIC;
        RegWrite  : in  STD_LOGIC;
        ReadData1 : out STD_LOGIC_VECTOR (31 downto 0);
        ReadData2 : out STD_LOGIC_VECTOR (31 downto 0)
    );
    end component;

    -- Testbench signals can have underscores, that is fine
    signal Read_reg1   : STD_LOGIC_VECTOR (4 downto 0) := (others => '0');
    signal Read_reg2   : STD_LOGIC_VECTOR (4 downto 0) := (others => '0');
    signal Write_reg   : STD_LOGIC_VECTOR (4 downto 0) := (others => '0');
    signal Write_data  : STD_LOGIC_VECTOR (31 downto 0) := (others => '0');
    signal clk         : STD_LOGIC := '0';
    signal RegWrite    : STD_LOGIC := '0';
    signal Read_data1  : STD_LOGIC_VECTOR (31 downto 0);
    signal Read_data2  : STD_LOGIC_VECTOR (31 downto 0);

    constant clk_period : time := 10 ns;

begin
    uut: RegisterFile PORT MAP (
        clk => clk,
        RegWrite => RegWrite,
        ReadReg1 => Read_reg1,
        ReadReg2 => Read_reg2,
        WriteReg => Write_reg,
        WriteData => Write_data,
        ReadData1 => Read_data1,
        ReadData2 => Read_data2
    );
```

```
clk_process :process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

stim_proc: process
    begin
        wait for 20 ns;
        -- Test Case 1: Write to Register 5
        RegWrite <= '1';
        Write_reg <= "00101";
        Write_data <= x"AAAAAAAA";
        wait for clk_period;

        -- Test Case 2: Write to Register 10
        Write_reg <= "01010"; -- Reg 10
        Write_data <= x"55555555";
        wait for clk_period;

        -- Test Case 3: Set RegWrite to zero then write in register 5
        RegWrite <= '0';
        Write_reg <= "00101";
        Write_data <= x"FFFFFFFF";
        wait for clk_period;

        -- Test Case 4: Read from Register 5 and 10 to verify previous
writes
        Read_reg1 <= "00101";
        Read_reg2 <= "01010";
        wait for clk_period;

        -- Test Case 5: Write and read in the same register in the same
clock cycle
        RegWrite <= '1';
        Write_reg <= "01111"; -- Reg 15
        Write_data <= x"BEEFCAFE";
        Read_reg1 <= "01111"; -- Read Reg 15 in the same cycle
        wait for clk_period;

        -- Test Case 6: Consecutive Writes and Simultaneous Reads
        RegWrite <= '1';
        Write_reg <= "10000"; -- Reg 16
        Write_data <= x"11112222";
        wait for clk_period;

        Write_reg <= "10001"; -- Reg 17
        Write_data <= x"33334444";
        wait for clk_period;
```

```
        RegWrite <= '0';
        Read_reg1 <= "10000";
        Read_reg2 <= "10001";
        wait for clk_period;

        -- Test Case 7: Rapid toggling of RegWrite
        RegWrite <= '1';
        Write_reg <= "11111"; -- Reg 31
        Write_data <= x"99999999";
        wait for clk_period;

        RegWrite <= '0';
        Write_reg <= "11111";
        Write_data <= x"00000000";
        wait for clk_period;

        Read_reg2 <= "11111";
        wait for clk_period;

        -- Test Case 8: Verify Register 0 is hardwired to zero
        RegWrite <= '1';
        Write_reg <= "00000";
        Write_data <= x"AAAAAAAA";
        wait for clk_period;

        RegWrite <= '0';
        Read_reg1 <= "00000";
        wait for clk_period;

        wait;
    end process;
end behavior;
```

✓ Repo Link

https://github.com/YassinAsaad07/Register-File-Implementation

2- Main points Explanations

1. Project Overview

A 32-register file implementation in VHDL with dual-port read and single-port write capability, commonly used in RISC processors (like MIPS architecture).

2. Register File Architecture (Register_File.vhd)

Key Components:

- **32 registers** × **32-bit width** storage array
- **Dual-port read**: Can read from two different registers simultaneously
- **Single-port write**: Can write to one register per clock cycle
- **Synchronous operation**: All operations synchronized to clock signal

2. Design Decisions:

- **Write on rising edge**: Data is written to registers on the positive clock edge when RegWrite = '1'
- **Read on falling edge**: Data is read on the negative clock edge to avoid read-write conflicts
- **5-bit addressing**: Supports 2^5 = 32 registers (addresses 0-31)
- **Initial state**: All registers initialized to zero
- **Constant Zero Register:** Register 0 is hardwired to strictly maintain a value of zero, preventing any overwrite operations.

3. Testbench Verification (tb_RegisterFile.vhd)

Test Coverage - 7 Test Cases:

1. **Basic Write Test**: Writes 0xAAAAAAAA to Register 5
2. **Multiple Writes**: Writes 0x55555555 to Register 10
3. **RegWrite Control**: Verifies write is disabled when RegWrite = '0'

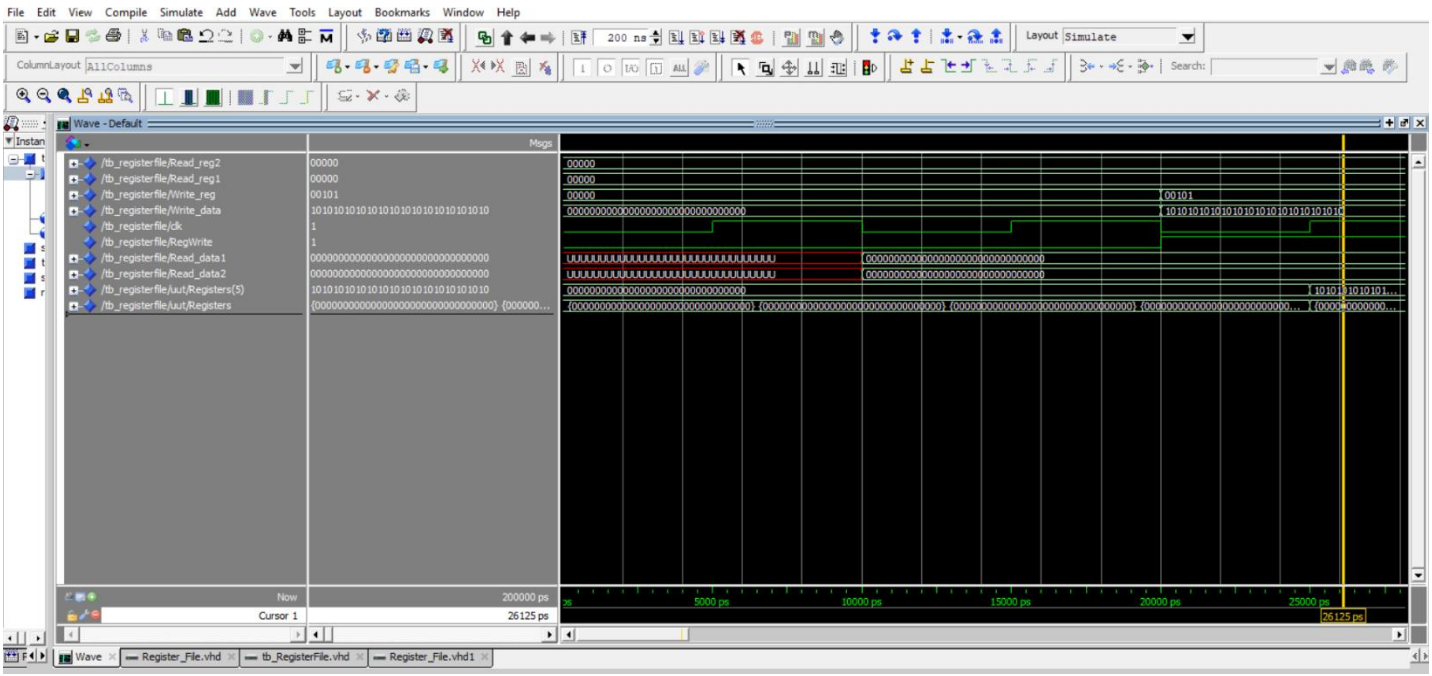
4. **Read Verification**: Confirms data integrity by reading Registers 5 and 10
5. **Same-Cycle Write/Read**: Tests simultaneous write and read to Register 15
6. **Consecutive Operations**: Multiple writes followed by dual reads (Registers 16-17)
7. **Control Signal Toggling**: Tests rapid RegWrite enable/disable transitions
8. **Constant Zero Verification:** Confirms Register 0 ignores write commands and maintains a strict zero value.

4. Technical Specifications

Parameter	**Value**
Number of Registers	32
Data Width	32 bits
Address Width	5 bits
Read Ports	2
Write Ports	1
Clock Period	10 ns
Special Features	Register 0 Hardwired to 0

3- Test Cases

Test Case 1 : Writes 0xAAAAAAAA to Register 5

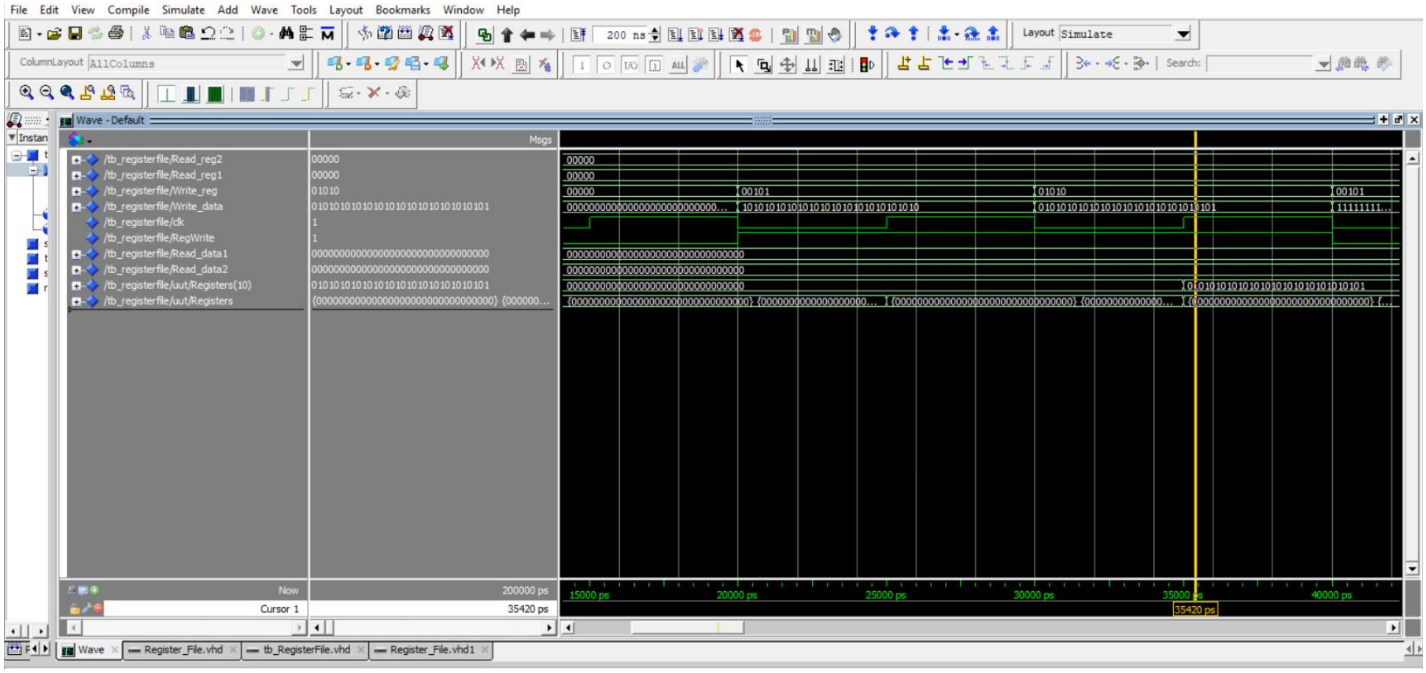


- This waveform proves the successful execution of **Test Case 1** and verifies the first half-cycle write requirement.
- At 20 ns, the testbench applies the stimulus, enabling RegWrite, targeting Register 5 (00101), and supplying the data AAAAAAAA (shown in binary).
- By observing the internal memory array signal uut/Registers(5), we can see it holds a value of zero until exactly 25 ns.
- On this exact rising edge of the clock, Registers(5) instantly updates to store the binary pattern 10101010... (AAAAAAAA), definitively proving that the write operation occurs correctly on the first half of the clock cycle.

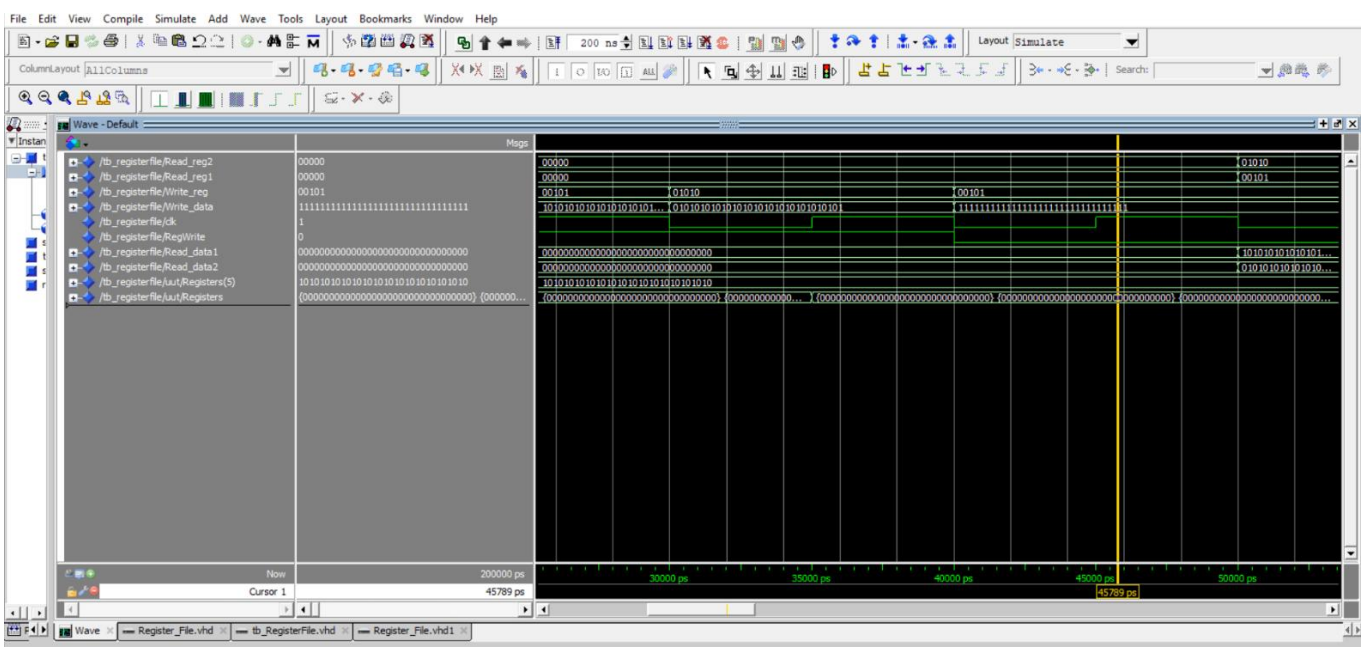
Test Case 2 : Writes 0x55555555 to Register 10

- At 30 ns, the testbench updates the inputs: Write_reg is set to Register 10 (01010), and Write_data is supplied with the binary pattern for 55555555 (01010101...).
- By monitoring the internal memory signal uut/Registers(10), we can observe that it remains zero until the clock transitions. Exactly at 35 ns, on the rising edge of the clock,

the data successfully latches into Registers(10). This further validates that the write operation correctly and consistently synchronizes with the first half of the clock cycle.



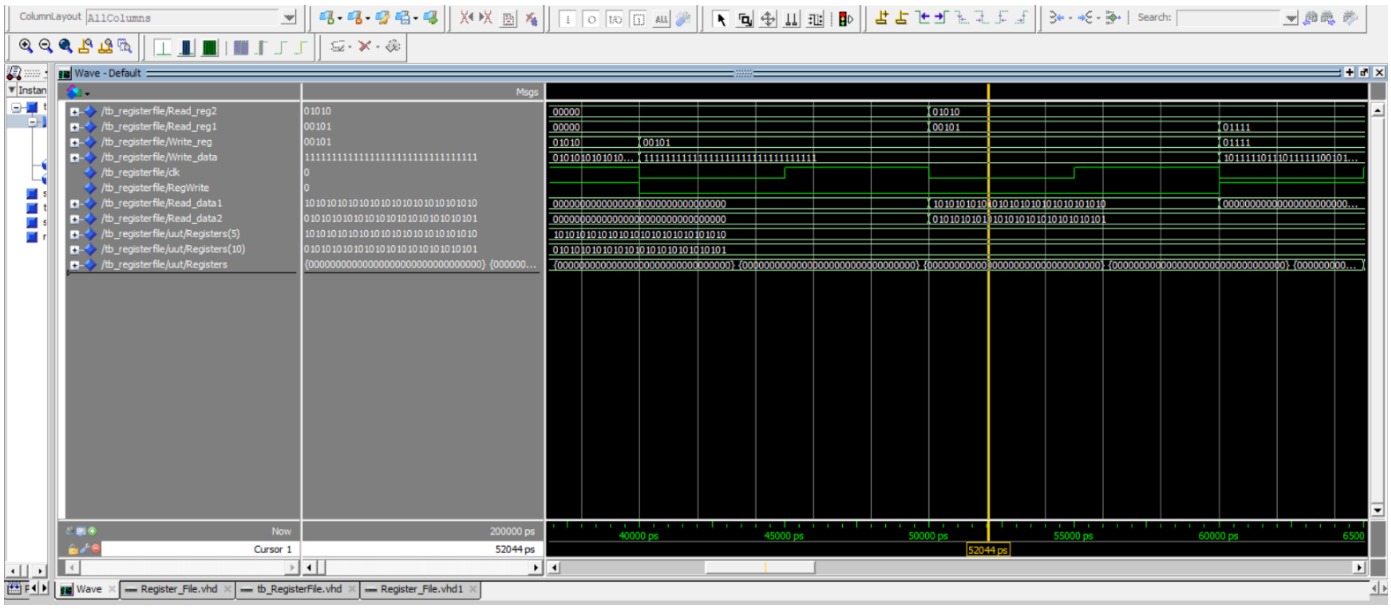
Test Case 3 : Verifies write is disabled when RegWrite = '0'



- At 40 ns, the testbench disables the write operation by setting RegWrite to '0', while simultaneously supplying a target address of Register 5 (00101) and new input data of FFFFFFFF (shown as all 1s). As the simulation passes the next rising clock edge at 45 ns, the internal memory signal uut/Registers(5) is monitored. Because RegWrite is disabled,

the register correctly ignores the new input data and safely retains its previously stored value of AAAAAAAA (10101010...). This definitively proves the write-control logic functions exactly as intended.

Test Case 4 : Confirms data integrity by reading Registers 5 and 10

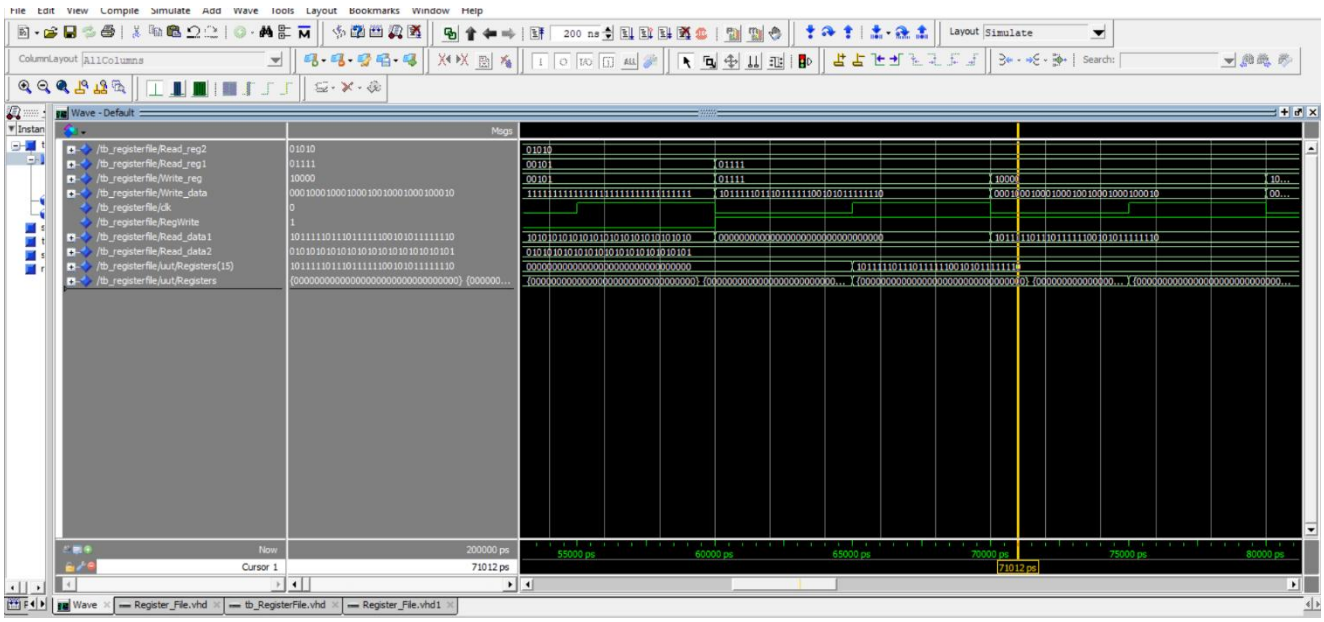


- Showing Read_data1 and Read_data2 successfully outputting the data on the falling edge at 50 ns

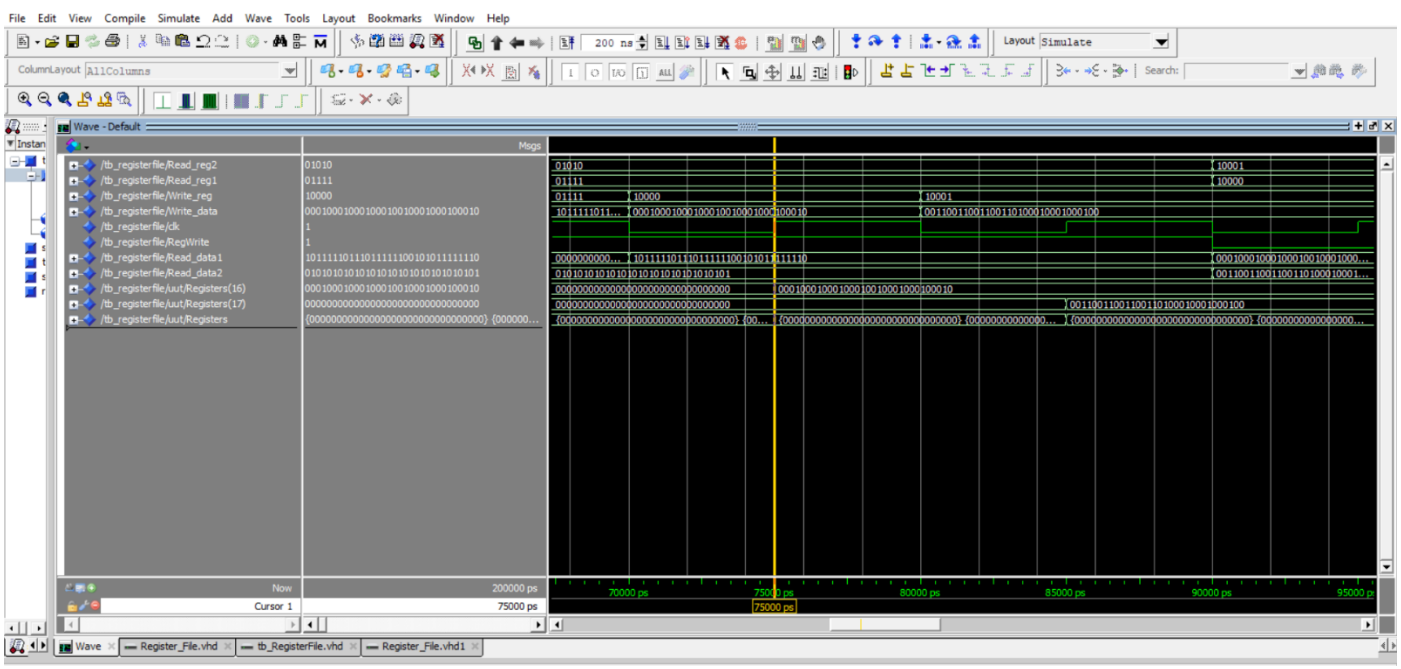
---

Test Case 5 : Tests simultaneous write and read to Register 15

- At 60 ns, the testbench sets both Write_reg and Read_reg1 to Register 15 (01111), enables RegWrite, and applies new data.
- By observing the internal memory signal uut/Registers(15), we can see the new data is successfully latched exactly on the rising edge of the clock at 65 ns (the first half-cycle).
- Subsequently, exactly on the falling edge of the clock at 70 ns (the second half-cycle), Read_data1 updates to output this newly stored data. This definitively proves the design successfully handles same-cycle write-then-read operations without data collision.



Test Case 6 : Multiple writes followed by dual reads (Registers 16-17)

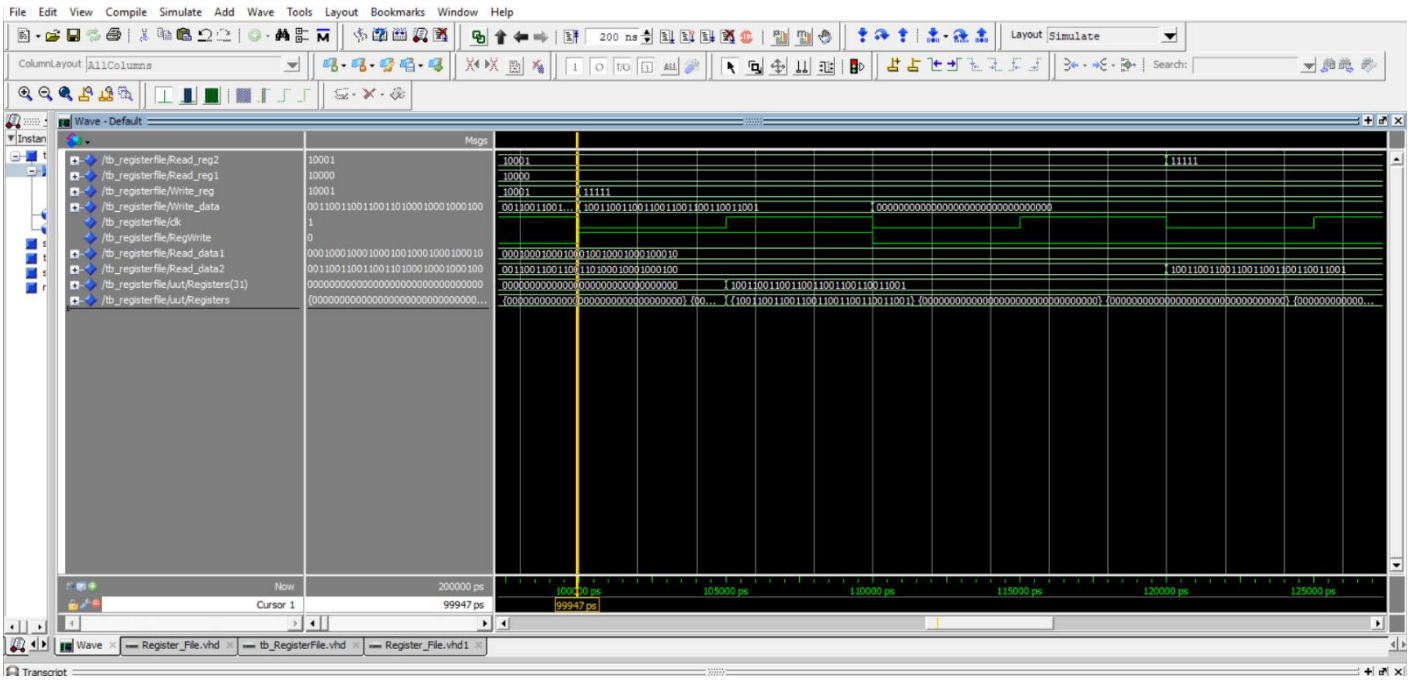


- This waveform verifies the register file's ability to handle consecutive write operations followed by simultaneous reads from independent ports.

- Starting at 70 ns, Write_reg targets Register 16 (10000), and the internal memory signal uut/Registers(16) successfully latches the data (11112222 in binary) on the 75 ns rising edge.
- In the next cycle at 80 ns, the target switches to Register 17 (10001), which similarly latches its new data (33334444 in binary) on the 85 ns rising edge.
- Finally, at 90 ns, RegWrite is disabled and both read ports are utilized simultaneously (Read_reg1 to 16, Read_reg2 to 17). Exactly on the 90 ns falling edge, Read_data1 and Read_data2 independently and correctly output the stored values, proving the two read ports operate flawlessly without interference

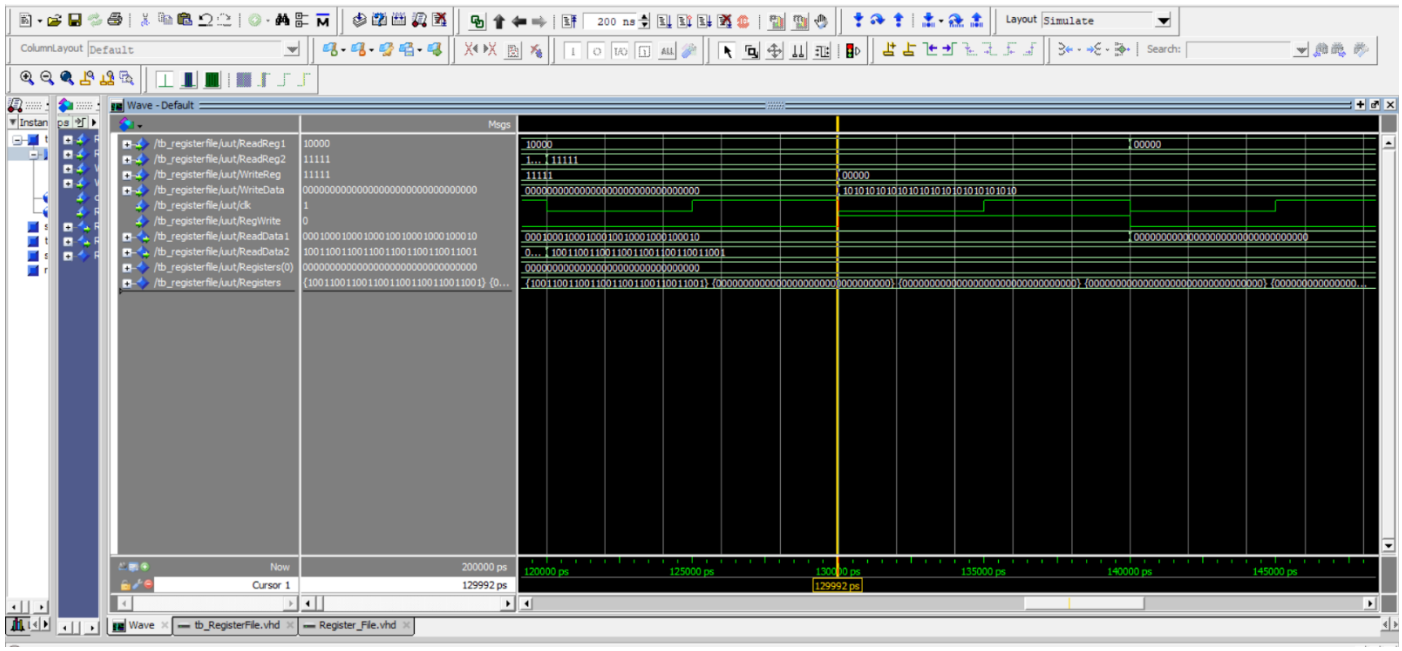
---

Test Case 7 : rapid RegWrite enable/disable transitions



- This waveform verifies the stability of the register file when the write-control signal is toggled rapidly. Starting at 90 ns, Write_reg targets Register 31 (11111) with RegWrite enabled ('1').
- The internal memory signal uut/Registers(31) successfully latches the data pattern (10011001... or hex 99999999) on the 95 ns rising clock edge.
- At 100 ns, RegWrite is deliberately disabled ('0'). Subsequently, the Write_data input is flooded with zeros. As the simulation passes the next rising edge at 115 ns, Registers(31) correctly ignores the zeros and maintains its stored value.
- Finally, on the 120 ns falling edge, Read_data2 successfully outputs the preserved data, proving the system is robust against data corruption when control signals toggle.

Test Case 8 : Constant Zero Verification



- This waveform verifies that Register 0 is hardwired to a constant zero and cannot be overwritten. Starting at 130 ns, WriteReg targets Register 0 (00000) with RegWrite enabled ('1') and a non-zero data pattern (10101010... or hex AAAAAAAA) on the input line.
- As the simulation passes the write-triggering rising clock edge at 135 ns, the internal memory signal uut/Registers(0) is actively monitored., the register successfully ignores the write command and maintains its strict zero value.
- Throughout this process and on subsequent falling edges (such as at 140 ns), ReadData1 confirms the register's contents remain entirely zero. This definitively proves the conditional logic correctly blocked the overwrite attempt, preserving Register 0's integrity.